

TD No1: Système d'exploitation

September 21, 2026

1 Noyau et appels système

1.1 Le noyau(kernel)

Question 1 Expliquez la notion d'architecture d'un processeur.

Question 2 Qu'est-ce que le noyau d'un OS.

1.2 Les appels systèmes

Question 1 Qu'est-ce qu'un appel système.

Question 2 Identifiez les différents appels systèmes produits par le code suivant (format 0x86_32), quelles routines du noyau sont invoquées.

```
.data
msg :      .asciz "Hello, World !\n"
len 15

.bss
.text
.global _start

_start :

    movl $4,%eax
    movl $1,%ebx
    movl $msg,%ecx
    movl $len,%edx
    int $0x80

exit :

    movl $1,%eax
    movl $0,%ebx
    int $0x80
```

Extrait du fichier `unistd.h` qui contient les macro des numéros d'appels système.

<code>#define</code>	<code>__NR_setup</code>	<code>0</code>	<code>/* used only by init , to get system going */</code>
<code>#define</code>	<code>__NR_exit</code>	<code>1</code>	
<code>#define</code>	<code>__NR_fork</code>	<code>2</code>	
<code>#define</code>	<code>__NR_read</code>	<code>3</code>	
<code>#define</code>	<code>__NR_write</code>	<code>4</code>	
<code>#define</code>	<code>__NR_open</code>	<code>5</code>	
<code>#define</code>	<code>__NR_close</code>	<code>6</code>	

Question 3 Compilez et exécutez ce fichier assembleur avec les directives suivantes :

```
as -o hello_32.o hello_32.s
ld -o hello_32 hello_32.o
./hello_32
```

Question 4 Exécutez la commande suivante:

```
strace ./hello_32
```

Que pouvez vous dire des résultats

Question 5 Modifiez le programme précédent pour écrire votre message dans un fichier **fic.txt** en ajoutant les appels systèmes **open** et **close** dans votre fichier.

Question 6 Créez un fichier **hello_64.s** qui utilise les registres préconisés sur une architecture 0x86_64. On trouve la description de ces registres dans le manuel de la fonction syscall.

```
man syscall
```

1.3 La LibC

Question 1 Décrivez le rôle de la LibC par rapport aux appels systèmes.

Question 2 Vous allez créer une mini LibC **mylibc.o** qui permettra à vos programmes de réaliser des appels systèmes avec des fonctions simple sans faire appel à la vrai LibC.

Pour cela vous allez créer les **wrapper** (fonctions qui contiennent le code assembleur nécessaire)

```
int my_write(int fd, const char *buf, int len)
{
    int ret;

    asm volatile (
        "movl $4, %%eax\n"
        "int $0x80\n"
        : "=a"(ret)
        : "b"(fd), "c"(buf), "d"(len)
        : "memory"
    );

    return ret;
}
```

Les directives de compilation sont les suivantes :

```
gcc -m32 -fno-pie -c mylibc.c -o mylibc.o
gcc -m32 -nostdlib -fno-pie -no-pie main.c mylibc.o -o main
```